new chunk to the stack for one of the sets of data and then sorted the other one directly. At this point, a straightforward wait for the sorting of the other chunk to complete would potentially deadlock, because you'd be consuming one of your limited number of threads waiting. It would be very easy to end up in a situation where all of the threads were waiting for chunks to be sorted and no threads were actually doing any sorting. We addressed this issue by having the threads pull chunks off the stack and sort them while the particular chunk they were waiting for was unsorted.

You'd get the same problem if you substituted a simple thread pool like the ones you've seen so far in this chapter instead of `std::async` in the example from chapter 4. There are now only a limited number of threads, and they might end up all waiting for tasks that haven't been scheduled because there are no free threads. You therefore need to use a solution similar to the one you used in chapter 8: process outstanding chunks while you're waiting for your chunk to complete. If you're using the thread pool to manage the list of tasks and their association with threads—which is, after all, the whole point of using a thread pool—you don't have access to the task list to do this. What you need to do is modify the thread pool to do this automatically.

The simplest way to do this is to add a new function on `thread_pool` to run a task from the queue and manage the loop yourself, so we'll go with that. Advanced thread pool implementations might add logic into the wait function or additional wait functions to handle this case, possibly prioritizing the task being waited for. The following listing shows the new `run_pending_task()` function, and a modified Quicksort to make use of it is shown in listing 9.5.

Listing 9.4. An implementation of `run_pending_task()`

```
void thread_pool::run_pending_task()
{
    function_wrapper task;
    if(work_queue.try_pop(task))
    {
        task();
    }
    else
    {
```